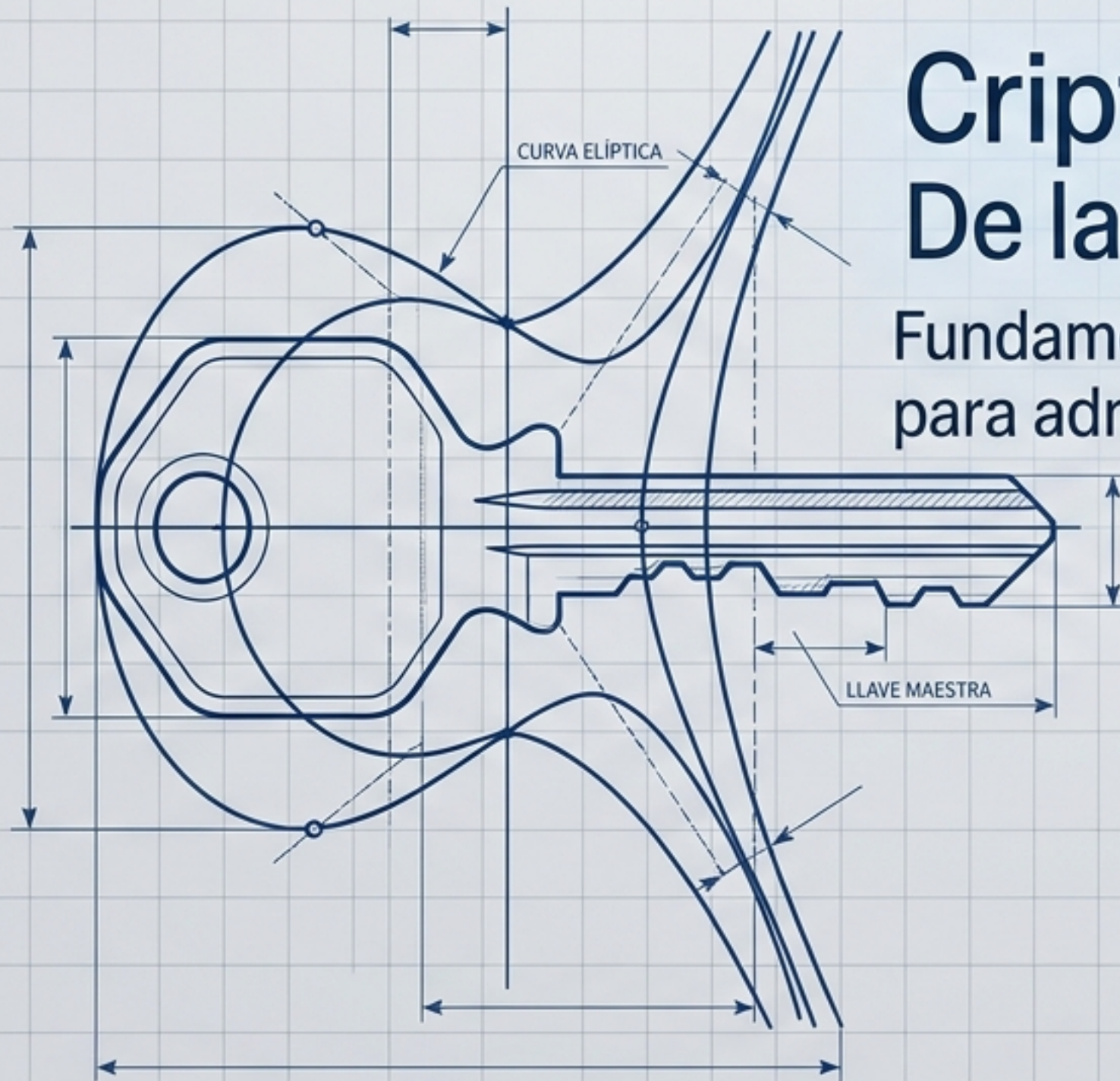




Criptografía Aplicada: De la Teoría a la Terminal

Fundamentos, implementación y buenas prácticas
para administradores de sistemas

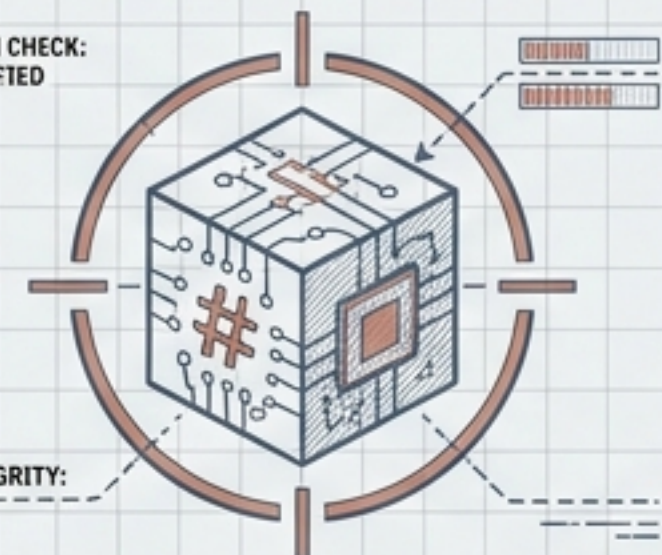
```
[root@server ~]$ openssl genrsa -out private_key.pem 2048
[root@server ~]$ openssl rsa -in private_key.pem -pubout -out
public_key.pem
[root@server ~]$ gpg --gen-key
[root@server ~]$ gpg --output encrypted_file.gpg --encrypt
--recipient admin@example.com secret_data.txt
[root@server ~]$ gpg --output decrypted_file.txt --decrypt
encrypted_file.gpg
[root@server ~]$
```



Confidencialidad

La información solo debe poder ser consultada por usuarios autorizados (ej. cifrado de discos o ficheros).

HASH CHECK:
VERIFIED



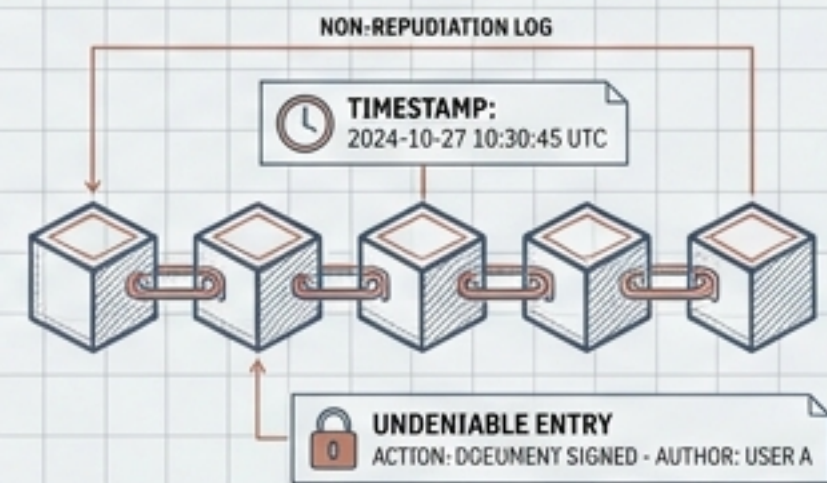
Integridad

Permite detectar modificaciones no autorizadas sobre la información (ej. comprobación de sumas hash).



Autenticidad

Permite comprobar la identidad del origen de una información (ej. firmas digitales).



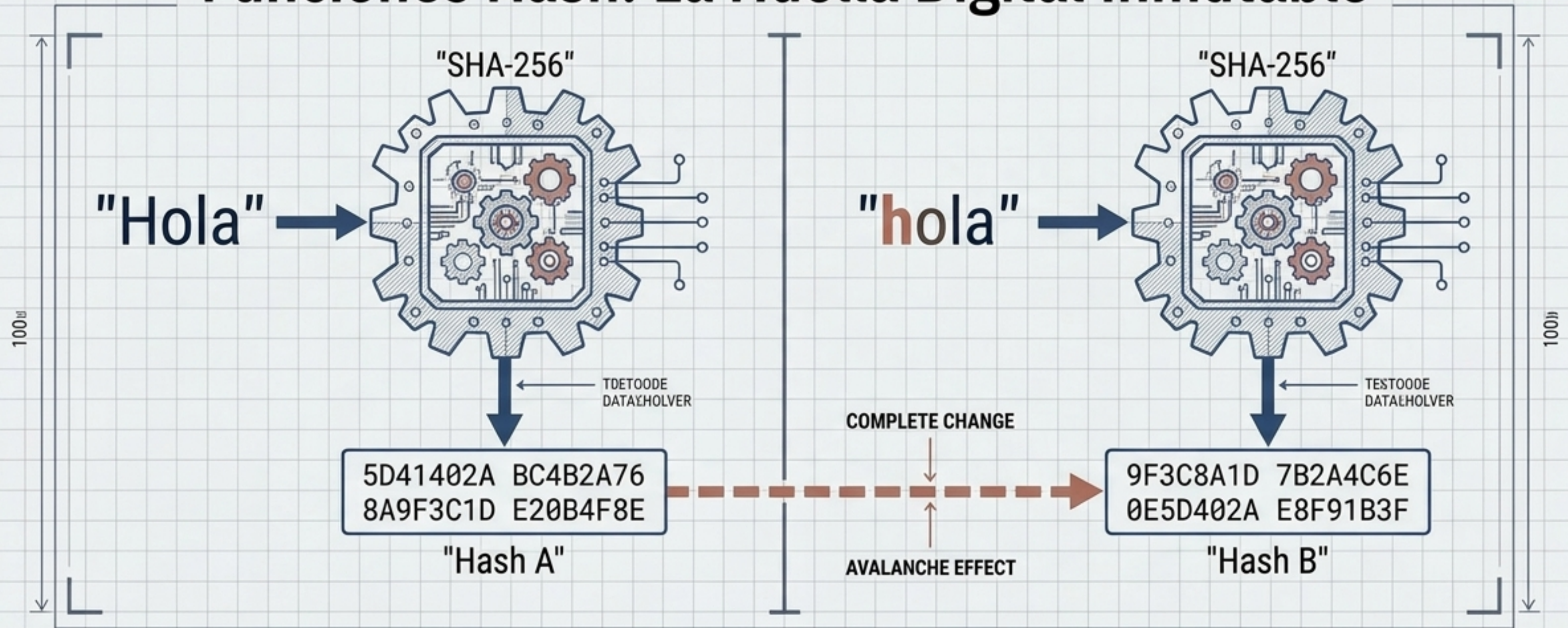
No repudio

Relaciona una determinada acción con su autor de forma innegable.

El Principio de Kerckhoffs

La seguridad no debe depender de mantener en secreto el algoritmo, sino de proteger correctamente la clave. Usa estándares públicos (AES, RSA) y protege tus claves.

Funciones Hash: La Huella Digital Inmutable



Unidireccional

Imposible obtener el dato original a partir del hash.



Sensibilidad extrema

Un cambio mínimo genera un resultado completamente distinto.



Longitud fija

SHA-256 siempre genera una huella del mismo tamaño, independientemente del archivo.

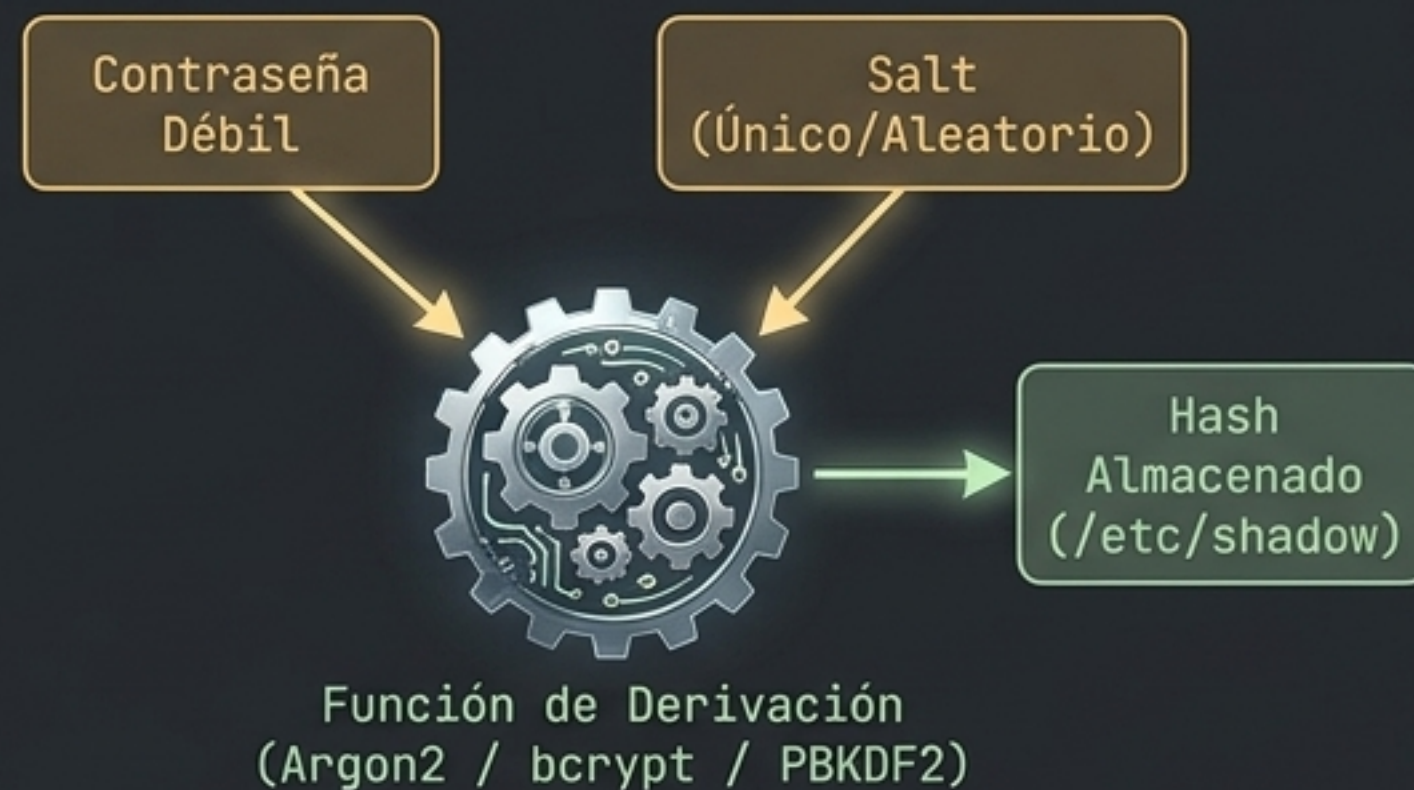
Práctica: Comprobación de Integridad y Contraseñas

Auditoría de Ficheros

```
sha256sum datos/mensaje.txt | tee  
datos/mensaje.txt.sha256  
  
sha256sum -c datos/mensaje.txt.sha256
```

📄 `sha256sum -c` compara la huella actual con la almacenada. Si falla, el fichero ha sido modificado.

Almacenamiento Seguro de Contraseñas



⚠️ **Nunca** almacenar hashes SHA-256 directos de contraseñas. Funciones como Argon2 encarecen ataques de fuerza bruta como los realizados con John the Ripper.

Matriz de Cifrado: Simétrico vs. Asimétrico

	Cifrado Simétrico	Cifrado Asimétrico
Arquitectura de Claves	1 Clave Secreta (Compartida)	2 Claves (Pública para distribuir, Privada para proteger)
Velocidad Computacional	Alta	Menor (Costosa computacionalmente)
Volumen de Datos Ideal	Muy adecuada para grandes cantidades de información (discos, payloads)	Menos adecuada (ideal para firmas o intercambio de claves)
Algoritmos Estándar	AES (128/192/256), ChaCha20	RSA, ECC (Curva Elíptica)

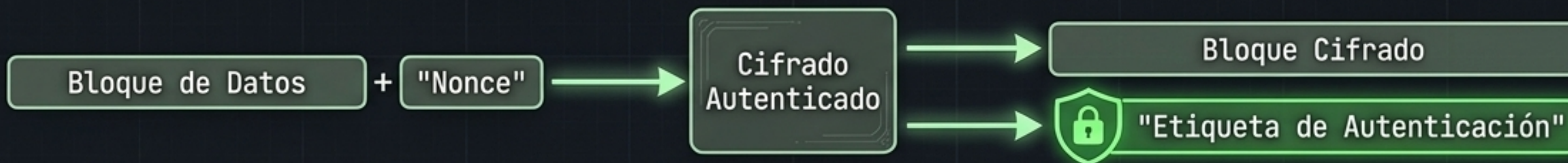
Cifrado Simétrico: Modos de Operación y Autenticación

Modo ECB (Riesgo Crítico)



Nunca usar para información sensible. Bloques iguales producen resultados iguales, revelando patrones ocultos.

Cifrado Autenticado (AES-GCM / ChaCha20-Poly1305)



Protege la confidencialidad y detecta modificaciones no autorizadas mediante una etiqueta.

La regla del Nonce (Vector de Inicialización)

No suele ser secreto, pero jamás debe reutilizarse con la misma clave si el algoritmo lo prohíbe.

Cifrado Asimétrico: La Dualidad de Claves

La Clave Pública



- Puede distribuirse libremente.
- Cifrar información destinada al propietario.
- Verificar firmas digitales.
- Participar en protocolos (Diffie-Hellman/ECDH).

La Clave Privada



- Debe mantenerse protegida celosamente.
- Descifrar información recibida.
- Generar firmas digitales.
- Autenticarse en servidores.

$f(x)$

Nota: La Criptografía de Curva Elíptica (ECC) ofrece el mismo nivel de seguridad que RSA, pero utilizando tamaños de clave drásticamente menores, optimizando el rendimiento.

Práctica: Operando con GnuPG

```
>_ Cifrado Simétrico Rápido - □ ×  
$ gpg --symmetric datos/confidencial.txt
```

Usa una frase de paso.
Si se olvida, los datos
son irrecuperables.

```
>_ Generación del Par Asimétrico - □ ×  
$ gpg --full-generate-key\  
Usuario: Lorena L Resusta <lorena.resusta@ejemplo.com>
```

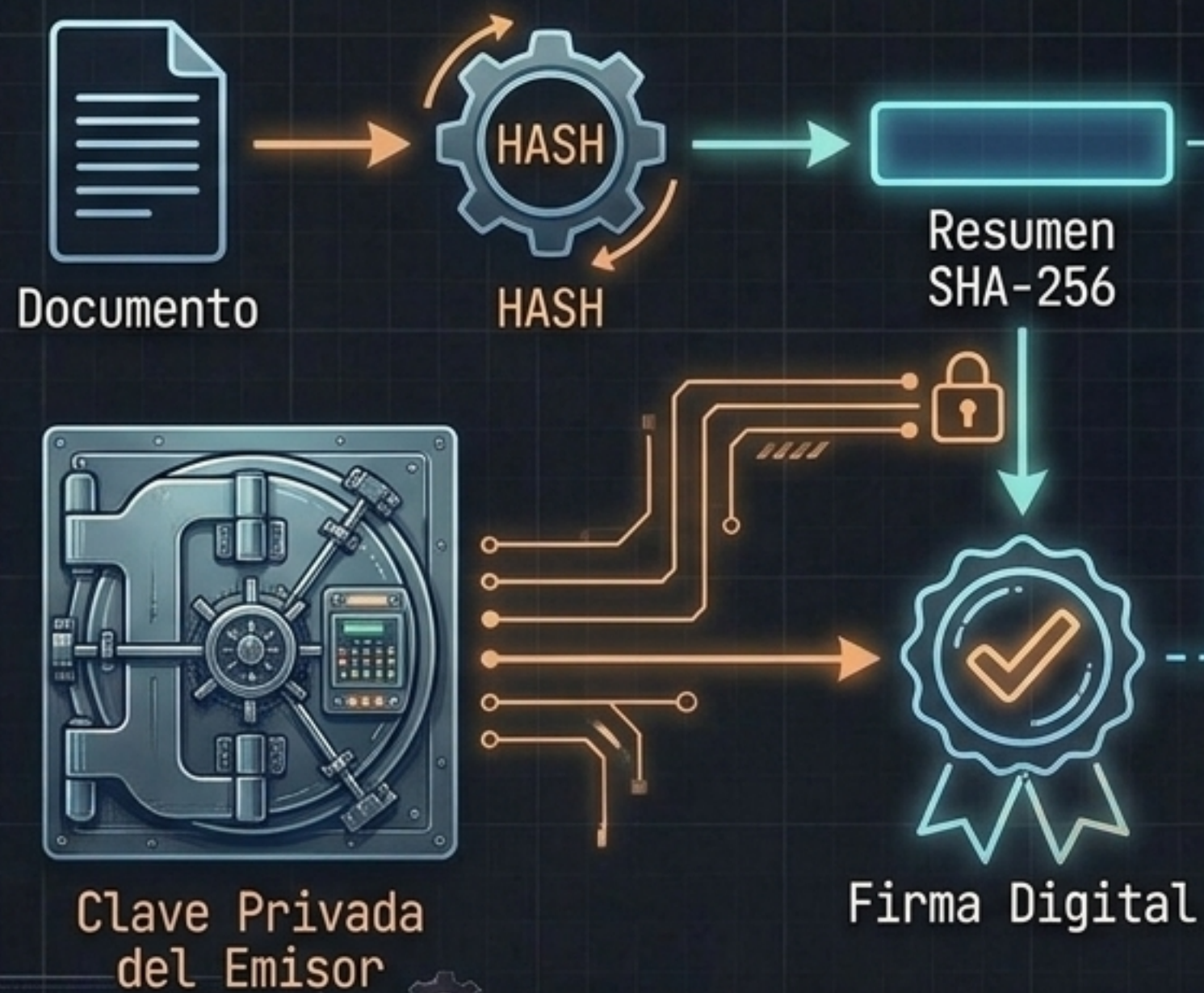
Crea las claves pública
y secreta en el anillo
de claves local.

```
>_ Cifrado y Firma Conjunta - □ ×  
$ gpg --armor --encrypt --sign --local-user  
"Lorena L Resusta" --recipient ID_PROFESOR  
mensaje.txt
```

`--armor` genera salida en texto ASCII. Protege la confidencialidad (cifrado) y garantiza el origen y la integridad (firma) simultáneamente.

Firma Digital: Garantizando Autoría e Integridad

El Emisor (Creación)



El Receptor (Verificación)



El Problema de la Confianza y el Certificado X.509



Formatos Comunes: PEM (Base64), DER (Binario), PKCS#12/PFX (Certificado + Privada).

Infraestructura de Clave Pública (PKI)



CA Raíz

Incluida de fábrica en navegadores y SO.



CA Intermedia

Protege la clave de la CA raíz y firma usuarios.



Certificado de Servidor

Ej. www.empresa.es

Gestión del Ciclo de Vida y Revocación

¿Qué ocurre si se compromete la clave privada o hay un error de emisión?

CRL

Listas estáticas de revocación de certificados.

OCSP

Protocolo de comprobación de estado en tiempo real.

Práctica: Desplegando una PKI con OpenSSL

Paso 1: Clave y Raíz de la CA

```
openssl genrsa -out ca.key 4096  
openssl req -x509 -new -key ca.key -sha256 -out ca.crt
```

Paso 2: Solicitud del Servidor (CSR)

```
openssl req -new -key servidor.key -out servidor.csr
```

La CSR contiene la clave pública del servidor y el nombre, pero NO la clave privada.

Paso 3: Firma de la CA y Extensión SAN

```
openssl x509 -req -in servidor.csr -CA ca.crt  
-CAkey ca.key -out servidor.crt
```

La extensión SAN es obligatoria para validar múltiples nombres DNS (ej. `www.servidorresusta.test`).

Síntesis: La Anatomía de TLS/HTTPS



El Horizonte: Criptografía Post-Cuántica (PQC)



Presente

Riesgo de captura hoy,
para descifrar mañana.
(Nota: AES y SHA-256 siguen
siendo robustos).

- ML-KEM (Establecimiento de claves)
- ML-DSA / SLH-DSA (Firmas digitales)

Amenaza: Algoritmo de Shor
(Ordenadores Cuánticos)

El Futuro / Estándares NIST

El Objetivo: Agilidad Criptográfica

Inventariar algoritmos actuales y mantener infraestructuras preparadas para adoptar las soluciones PQC estandarizadas de forma fluida.

El Manifiesto del Administrador de Sistemas

- 1** **Protección de la Privada:** Nunca compartas ni envíes claves privadas. Usa permisos estrictos (chmod 600).
- 2** **No inventes algoritmos:** Usa estándares públicos (AES, RSA, ECC). La seguridad reside en la clave (Principio de Kerckhoffs).
- 3** **Huye del modo ECB:** Utiliza modos de cifrado autenticado (AES-GCM o ChaCha20-Poly1305) y gestiona correctamente los nonces.
- 4** **Hashes solos no bastan:** Protege las contraseñas con funciones de derivación robustas (Argon2, bcrypt) combinadas con Salts únicos.
- 5** **Practica la agilidad:** Prepárate para rotar claves y certificados comprometidos rápidamente ante la llegada de la era post-cuántica.

ADMIN_SYS# _